HDOC Day-58

Finding the Kth Smallest Element Using Bubble Sort, Functions, and Pointers in C

Aim

To write a C program using a variable-length integer array, pointers, a user-defined function, and Bubble Sort to find the kth smallest element, counting duplicate elements separately. Display -1 for an invalid value of k.

Algorithm

1. Start the program and read the array size. Reject non-positive sizes.
2. Declare the variable-length integer array arr[size] and read its elements.
3. Read k and pass the array, size, and k to kth_smallest() using its address.
4. Inside the function, if k is not between 1 and size inclusive, display -1 and return.
5. Use nested loops to compare adjacent elements and swap them when they are out of ascending order.
6. Display the sorted array, including duplicates.
7. Display the element at index k - 1 as the kth smallest element.
8. Stop the program.

C Program

```
#include <stdio.h>

void kth_smallest(int *arr, int size, int k);

int main()
{
    int size, k, i;

    printf("Enter the size of array: ");
    scanf("%d", &size);

    if (size <= 0)
    {
        printf("Invalid array size!\n");
        return 0;
    }

    int arr[size];

    printf("Enter %d array elements: ", size);
    for (i = 0; i < size; i++)
    {
        scanf("%d", &arr[i]);
    }

    printf("Enter the value of k: ");
    scanf("%d", &k);

    kth_smallest(arr, size, k);
    return 0;
}

/* Function to find the kth smallest element */
void kth_smallest(int *arr, int size, int k)
{
    int i, j, temp;

    if (k <= 0 || k > size)
    {
        printf("Kth Smallest Element: -1\n");
        return;
    }

    /* Bubble Sort in ascending order */
    for (i = 0; i < size - 1; i++)
    {
        for (j = 0; j < size - i - 1; j++)
        {
            if (*(arr + j) > *(arr + j + 1))
            {
                temp = *(arr + j);
                *(arr + j) = *(arr + j + 1);
                *(arr + j + 1) = temp;
            }
        }
    }

    printf("\nSorted Array: ");
    for (i = 0; i < size; i++)
    {
        printf("%d ", *(arr + i));
    }
    printf("\n");

    printf("Kth Smallest Element: %d\n", *(arr + k - 1));
}
```

Explanation

The function kth_smallest(arr, size, k) receives the array through a pointer. The expression *(arr + i) is equivalent to arr[i]. Bubble Sort repeatedly swaps adjacent elements until the array is in ascending order. Since C uses zero-based indexing, the kth smallest element is at index k - 1. Duplicates are not removed.

Example

```
Input:  7 10 4 3 20 15
size = 6, k = 3
Sorted: 3 4 7 10 15 20
Kth Smallest Element: 7
```

Test Case Table

Case	Input Array	k	Expected Result
1	7 10 4 3 20 15	3	7
2	1 2 3 4 5	2	2
3	5 4 3 2 1	4	4
4	7 7 4 3 7 2	4	7
5	-5 -2 -8 -1 -4	2	-5
6	10 20 30 40	4	40
7	4 2 7 1 9	1	1
8	1 2 3 4 5	0	-1
9	1 2 3 4 5	6	-1
10	size = 0	-	Invalid array size

Sample Outputs

Kth smallest element

```
Enter the size of array: 6
Enter 6 array elements: 7 10 4 3 20 15
Enter the value of k: 3

Sorted Array: 3 4 7 10 15 20
Kth Smallest Element: 7
```

Duplicate elements

```
Enter the size of array: 6
Enter 6 array elements: 7 7 4 3 7 2
Enter the value of k: 4

Sorted Array: 2 3 4 7 7 7
Kth Smallest Element: 7
```

Negative elements

```
Enter the size of array: 5
Enter 5 array elements: -5 -2 -8 -1 -4
Enter the value of k: 2

Sorted Array: -8 -5 -4 -2 -1
Kth Smallest Element: -5
```

Invalid k

```
Enter the size of array: 5
Enter 5 array elements: 1 2 3 4 5
Enter the value of k: 6
Kth Smallest Element: -1
```

Invalid array size

```
Enter the size of array: 0
Invalid array size!
```

Result

The C program was designed using variable-length arrays, pointers, a user-defined function, nested loops, conditional statements, and Bubble Sort. It sorts the input array, counts duplicate elements separately, and displays the kth smallest element or -1 when k is invalid.